

# EdgeStyleGAN: Lightweight and Efficient StyleGAN for Real-Time Image Synthesis

Jash Shah - 202201016@dau.ac.in

Dhirubhai Ambani University

December 1, 2025

# Overview

1. Introduction
2. Background
3. MobileStyleGAN Architecture
4. Our Approach
5. Results
6. Applications
7. Conclusion

# Abstract

Generative Adversarial Networks (GANs), and particularly StyleGAN, have demonstrated state-of-the-art performance in high-fidelity image synthesis. However, the computational and memory demands of StyleGAN severely restrict its applicability in resource-constrained environments such as mobile and edge devices. MobileStyleGAN addressed this through knowledge distillation and DWT-based architecture, achieving 68.7% parameter reduction. However, the authors left pruning and quantization for future exploration. In this work, we present EdgeStyleGAN, which further optimizes MobileStyleGAN through unstructured pruning and static quantization. Our approach systematically reduces the model size and inference latency. Experimental results demonstrate that EdgeStyleGAN achieves 56.2% further parameter reduction (from 8.85M to 3.88M) and  $2.28\times$  compression ratio while maintaining reasonable perceptual quality with FID score of 16.57. This work completes the compression pipeline, achieving 86.3% total parameter reduction from StyleGAN2, highlighting a practical pathway for democratizing GAN-based content generation on low-power devices.

# Why StyleGAN?

## State-of-the-Art Image Synthesis

- **Revolutionary quality:** Photorealistic faces at  $1024 \times 1024$  resolution
- **FID score:** 2.84 on FFHQ dataset, near-perfect visual fidelity
- **Versatile applications:** Faces, artwork, design, fashion, avatars

## Key Innovations

- Style-based architecture with latent space control
- Produces images indistinguishable from real photographs
- Fine-grained control over image attributes (pose, lighting, features)

## The Challenge

**Computational barrier:** Even MobileStyleGAN requires significant resources (8.85M parameters, 33.78 MB model size) for mobile deployment

# The Problem: Deployment Barrier

## Hardware Constraints

- Mobile/edge devices have no dedicated GPUs
- Limited RAM and battery life
- ARM processors vs. NVIDIA CUDA cores

## MobileStyleGAN Baseline

- 8.85M parameters
- 33.78 MB model size
- Still requires optimization for edge devices

## Real-World Impact

- **AR/VR personalization:** Real-time avatar generation blocked
- **Mobile creative tools:** Cannot run on smartphones/tablets
- **Edge AI:** IoT devices and embedded systems excluded
- **Latency:** Server-based inference adds network delays

# Motivation: Why Mobile Deployment Matters

- **Privacy:** On-device generation, no data sent to cloud servers
- **Latency:** Real-time generation for interactive applications
- **Cost:** Eliminate expensive GPU cloud infrastructure
- **Accessibility:** Democratize AI art and personalization tools
- **Scalability:** Billions of mobile devices worldwide
- **Offline capability:** Works without internet connectivity

# MobileStyleGAN: Knowledge Distillation Approach

## Teacher-Student Paradigm

- **Teacher:** Pre-trained StyleGAN2 (frozen, 28.27M parameters)
- **Student:** MobileStyleGAN (trainable, 8.85M parameters)
- Student learns to mimic teacher outputs through knowledge distillation

## Key Architectural Changes

- **Depthwise Convolutions:** Replace standard convolutions to reduce parameters
- **DWT-based Upsampling:** Use Discrete Wavelet Transform instead of bilinear interpolation
- **Frequency Domain Processing:** Operate in frequency domain for efficiency
- **Shared Mapping Network:** Same mapping network as teacher

## Future Work

MobileStyleGAN authors left **pruning** and **quantization** for future exploration

# DWT: Discrete Wavelet Transform

## What is DWT?

Mathematical transform that decomposes images into frequency components (low-pass and high-pass coefficients)

### DWT Forward (Analysis)

- Decomposes image into:
- 1 low-frequency component (LL)
- 3 high-frequency components (LH, HL, HH)
- $4\times$  fewer parameters per channel

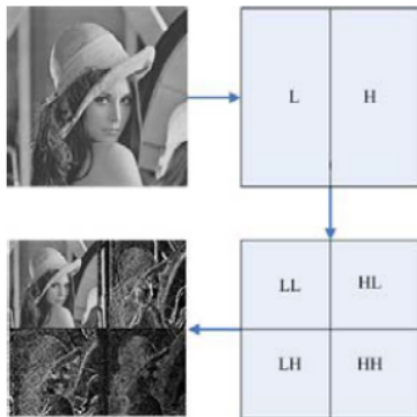
### DWT Inverse (Synthesis)

- Reconstructs image from coefficients
- Efficient upsampling ( $2\times$  resolution)
- Preserves image quality
- Hardware-friendly operations

## In MobileStyleGAN

Each block outputs 12 channels: 3 low-frequency + 9 high-frequency ( $3\times 3$ ) coefficients. Final DWT inverse converts to RGB.

# DWT Decomposition Example



**Figure:** Single-level DWT decomposition: original image is split into low-frequency (LL) and high-frequency (LH, HL, HH) subbands capturing coarse structure and edge details.

# MobileStyleGAN Architecture Details

## Mobile Synthesis Network Structure

1. **Constant Input:** Initial constant tensor (512 channels)
2. **Initial Styled Convolution:** Style-modulated conv layer
3. **Multichannel Image:** Converts to frequency domain (12 channels)
4. **Mobile Synthesis Blocks** (7 blocks):
  - IDWT Upsample:  $2\times$  upsampling via DWT inverse
  - Two Styled Convolutions with depthwise conv
  - Multichannel Image: Generate frequency representation
5. **Final DWT Inverse:** Convert frequency domain to RGB image

## Channel Progression

512  $\rightarrow$  512  $\rightarrow$  512  $\rightarrow$  512  $\rightarrow$  512  $\rightarrow$  256  $\rightarrow$  128  $\rightarrow$  64

## Key Innovation

Operating in frequency domain allows efficient processing while maintaining image quality.

# Prediction Head: StyleGAN2 vs MobileStyleGAN

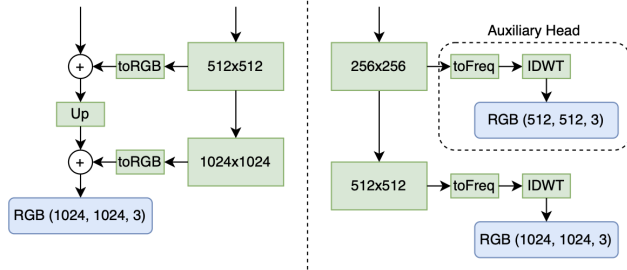


Figure 3. (left) Prediction head of StyleGAN2. (right) Prediction head of MobileStyleGAN.

Figure: Prediction head comparison: StyleGAN2 (left) vs MobileStyleGAN (right)

## Key Differences

- **StyleGAN2:** Uses direct RGB conversion ( $\text{toRGB}$ ) with bilinear upsampling ( $\text{Up}$ ) and skip connections
- **MobileStyleGAN:** Uses frequency domain conversion ( $\text{toFreq}$ ) with IDWT for upsampling and reconstruction
- **Efficiency:** IDWT inherently performs  $2\times$  upsampling ( $512\times 512 \rightarrow 1024\times 1024$ ) while operating in frequency domain
- **Auxiliary Head:** MobileStyleGAN includes an auxiliary head producing  $512\times 512$  RGB output for multi-scale training

# Modulated Convolution Architectures

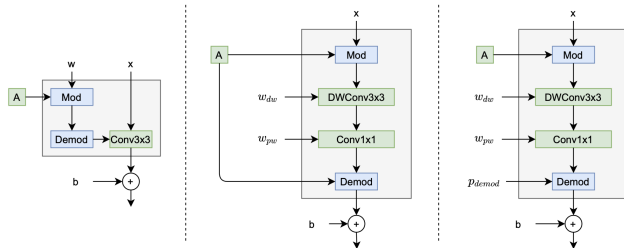


Figure 4. (left) Modulated convolution. (middle) Depthwise separable modulated convolution. (right) Depthwise separable modulated convolution with a trainable demodulation.

**Figure:** Evolution of modulated convolution: Standard (left), Depthwise Separable (middle), Trainable Demodulation (right)

## Architecture Evolution

- **Left:** Standard modulated convolution with  $3 \times 3$  conv, modulation/demodulation on weights
- **Middle:** Depthwise separable modulated conv (DWConv $3 \times 3$  + Conv $1 \times 1$ ) for efficiency
- **Right:** Trainable demodulation parameter ( $p_{\text{demod}}$ ) instead of direct style code connection

## MobileStyleGAN Innovation

Uses depthwise separable convolutions to reduce parameters while maintaining style modulation capabilities. The trainable demodulation allows more flexible feature transformation.

# Upscale Block: StyleGAN2 vs MobileStyleGAN

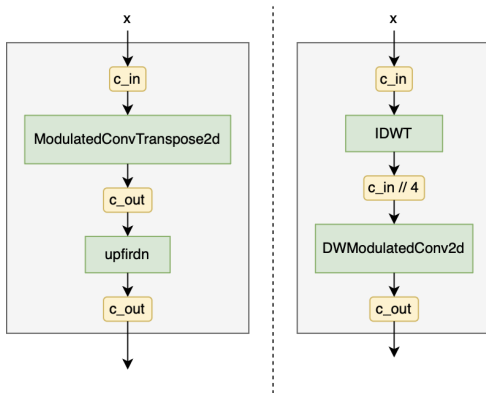


Figure 5. (left) StyleGAN2 upscale block. (right) MobileStyleGAN upscale block. Yellow rectangles on arrows show the number of channels in related feature maps.

## StyleGAN2 Upscale Block

- Uses `ModulatedConvTranspose2d` for upsampling
- Followed by `upfirdn` (upsampling, filtering, downsampling) operation
- Channel count:  $c_{in} \rightarrow c_{out}$  (no reduction)

## MobileStyleGAN Upscale Block

- Uses IDWT (Inverse Discrete Wavelet Transform) for upsampling
- Channel reduction:  $c_{in} \rightarrow c_{in}/4$  ( $4\times$  reduction after IDWT)
- Followed by `DWModulatedConv2d` (depthwise modulated convolution)
- Final output:  $c_{out}$  channels

## Efficiency Gain

IDWT provides  $2\times$  spatial upsampling while reducing channels by  $4\times$ , significantly reducing computational cost compared to transpose convolution.

# Building Block: StyleGAN2 vs MobileStyleGAN

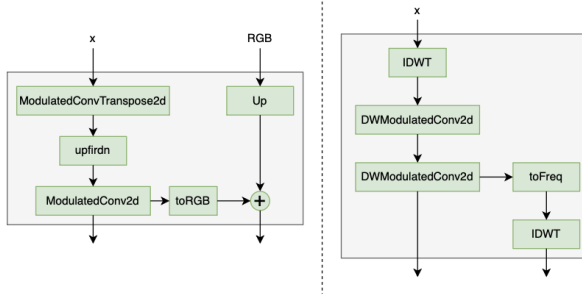


Figure 6. (left) StyleGAN2 building block. (right) MobileStyleGAN building block.

Figure: Building block comparison: StyleGAN2 (left) vs MobileStyleGAN (right)

## StyleGAN2 Block

- ModulatedConvTranspose2d + upfirdn
- ModulatedConv2d
- toRGB conversion
- Skip connection with Up operation

## MobileStyleGAN Block

- IDWT upsampling
- Two DWModulatedConv2d layers
- toFreq conversion
- Skip connection with IDWT

### Key Architectural Changes

- **Convolution:** Standard  $\rightarrow$  Depthwise separable (reduces parameters)
- **Upsampling:** Transpose conv + upfirdn  $\rightarrow$  IDWT (more efficient)
- **Domain:** RGB spatial  $\rightarrow$  Frequency domain (DWT coefficients)
- **Skip Connection:** RGB addition  $\rightarrow$  Frequency domain addition

# Training Framework: Knowledge Distillation

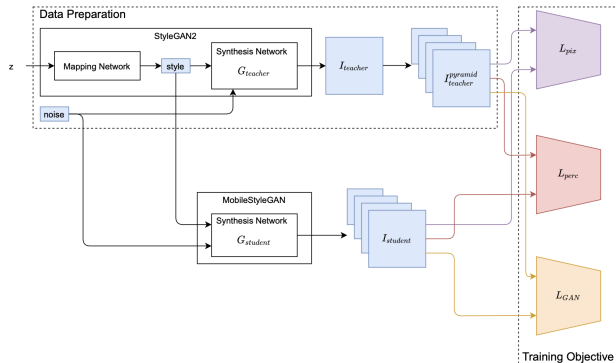


Figure 7. Training framework.

Figure: Knowledge distillation training framework for MobileStyleGAN

## Training Pipeline

1. **Data Preparation:** Latent code  $z \rightarrow$  Mapping Network  $\rightarrow$  Style vector + Noise
2. **Teacher (StyleGAN2):** Generates  $I_{teacher}$  and multi-resolution pyramid  $I_{teacher}^{pyramid}$
3. **Student (MobileStyleGAN):** Generates  $I_{student}$  with multi-resolution features
4. **Loss Computation:** Three loss functions guide student training

## Loss Functions

- $L_{pix}$ : Pixel-wise loss comparing teacher and student outputs at multiple resolutions
- $L_{perc}$ : Perceptual loss using VGG features for semantic similarity
- $L_{GAN}$ : Adversarial loss ensuring student generates realistic images

# MobileStyleGAN vs StyleGAN2: Architectural Comparison

| Feature               | StyleGAN2              | MobileStyleGAN                   |
|-----------------------|------------------------|----------------------------------|
| Parameters            | 28.27M                 | 8.85M (68.7% reduction)          |
| Computation (GMAC)    | 143.15                 | 15.09 (89.5% reduction)          |
| Upsampling Method     | Bilinear interpolation | DWT inverse                      |
| Convolution Type      | Standard conv          | Depthwise conv                   |
| Domain                | Spatial (RGB)          | Frequency (DWT coefficients)     |
| Training Method       | Standard GAN training  | Knowledge distillation           |
| FID (FFHQ, 1024×1024) | 2.84                   | 7.75                             |
| Inference Speed (CPU) | 4.3s                   | 1.2s (PyTorch), 0.16s (OpenVINO) |
| Model Size            | Large                  | 33.78 MB (checkpoint: 722.5 MB)  |

Table: Architectural and performance comparison: StyleGAN2 vs MobileStyleGAN

## MobileStyleGAN Achievements

- **68.7%** parameter reduction from StyleGAN2
- **3.6×** faster inference on CPU

## Building on MobileStyleGAN

MobileStyleGAN authors left **pruning** and **quantization** for future work. We explore these optimizations to further compress the model.

Two key optimizations:

- **Unstructured Pruning:** remove individual weights based on importance.
- **Static Quantization:** reduce numerical precision from FP32 to INT8.

*Further compressing MobileStyleGAN for edge deployment.*

# Optimization 1: Unstructured Pruning

## What is Unstructured Pruning?

Remove individual weights (not entire channels/filters) based on importance, creating sparse weight matrices

### How it Works:

- Analyze weight magnitude/importance
- Remove low-magnitude weights
- Create sparse weight matrices
- Fine-tune remaining weights

### Benefits:

- Higher sparsity possible
- Faster inference with sparse support
- Flexible pruning granularity
- Better parameter reduction

## Pruning Amount → Sparsity & FID

0.2 → 16.63% (FID: 11.8868)

0.4 → 33.26% (FID: 13.5677)

0.6 → 49.88%

0.8 → 66.51%

# Unstructured Pruning: Visual Results



20% pruning  
Sparsity: 16.63%  
FID: 11.8868



40% pruning  
Sparsity: 33.26%  
FID: 13.5677

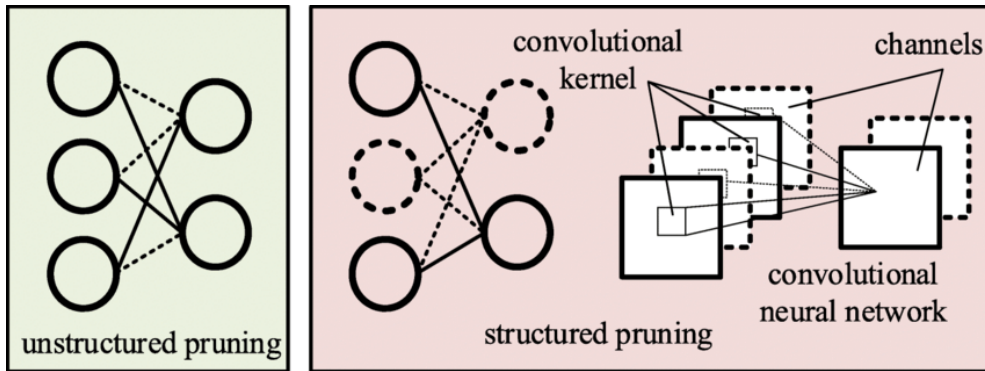


60% pruning  
Sparsity: 49.88%



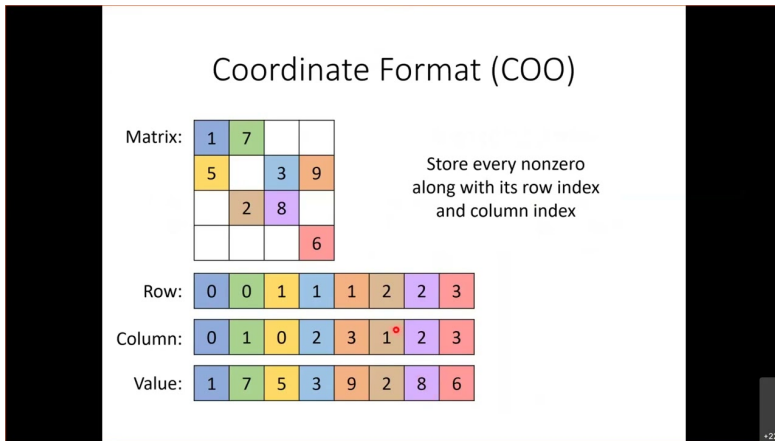
80% pruning  
Sparsity: 66.51%

# Structured vs Unstructured Pruning



**Figure:** Structured vs Unstructured Pruning: A high-level view of how pruning functions in compressing deep neural networks. Traditional pruning schemes can be divided into two classes, that is, unstructured pruning that simply removes inter-layer connections between neurons while structured pruning removes neurons as well as their connected weights, or channels with their corresponding kernels w.r.t. the convolutional neural network

# Sparse Matrix Storage: COO Format



**Figure:** Coordinate Format (COO): A sparse matrix representation that stores every nonzero value along with its row index and column index

# Optimization 2: Static Quantization

## What is Static Quantization?

Reduce numerical precision from FP32 to INT8 for both weights and activations, calibrated before inference

### Our Implementation:

- Method: Static Quantization
- Backend: QNNPACK (ARM optimized)
- FP32  $\rightarrow$  INT8 ( $4\times$  smaller)
- Compression:  $2.28\times$  ratio

### Results:

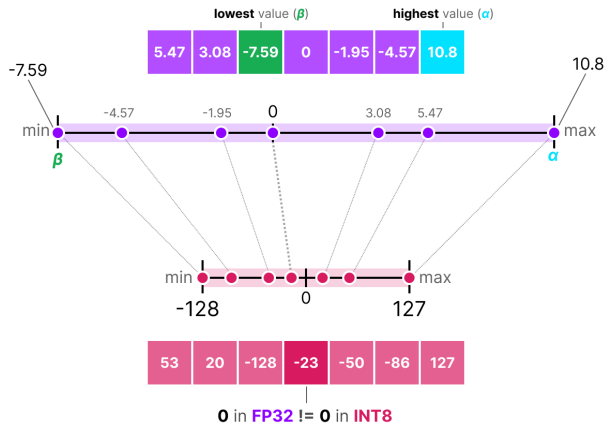
- Model size: 33.78 MB  $\rightarrow$  14.81 MB
- Parameters: 8.85M  $\rightarrow$  3.88M (56.2% reduction)
- FID Score: 16.57
- Optimized for mobile/edge devices

## QNNPACK Backend

Optimized for ARM processors (phones, tablets, edge devices) using ARM NEON instructions

**Advantage:** Full quantization (weights + activations) for maximum speedup

# Quantization Process



**Figure:** Quantization Process: Mapping FP32 values to INT8 representation. Note that zero in FP32 does not necessarily map to zero in INT8 after quantization

# Pruning vs Quantization Comparison

| Feature          | Unstructured         | Structured       | Sparse Format |
|------------------|----------------------|------------------|---------------|
| File Size        | No reduction         | Reduces          | Reduces       |
| Inference Speed  | Faster               | Faster           | Same          |
| Hardware Support | Needs sparse support | Works everywhere | Storage only  |

Table: Comparison of pruning strategies

| Feature                   | Dynamic                        | Static                         |
|---------------------------|--------------------------------|--------------------------------|
| When quantization happens | At runtime                     | At calibration time            |
| Calibration data needed   | No                             | Yes                            |
| Weights                   | Quantized (int8)               | Quantized (int8)               |
| Activations               | Float32                        | Quantized (int8)               |
| Speed                     | Faster setup, slower inference | Slower setup, faster inference |
| Accuracy                  | Lower                          | Higher                         |
| Best for                  | Models with many Linear layers | Production deployment          |

Table: Static vs Dynamic Quantization

# Results Summary

## Original Model

- Size: 33.78 MB (checkpoint: 722.5 MB)
- Parameters: 8,852,079 (all trainable)

## After Unstructured Pruning

- Pruned models: 35.5 MB (20%, 40%, 60%, 80% sparsity)
- Sparsity levels: 16.63%, 33.26%, 49.88%, 66.51%

## After Static Quantization (QNNPACK)

- Quantized size: 14.81 MB (checkpoint: 20.7 MB)
- Compression ratio: **2.28×**
- Parameters: 3,878,031 (**56.2% reduction**)
- FID Score: 16.57

# Sample Image



Figure 1. (top) Images generated by StyleGAN2. (bottom) Images generated by MobileStyleGAN.

Figure: Generated samples using MobileStyleGAN.

# MobileStyleGAN vs EdgeStyleGAN: Compression Comparison

| Metric                 | MobileStyleGAN   | EdgeStyleGAN (Quantized)        |
|------------------------|------------------|---------------------------------|
| Parameters             | 8,852,079        | 3,878,031                       |
| Parameter Reduction    | Baseline         | <b>56.2% reduction</b>          |
| Model Size (in-memory) | 33.78 MB         | 14.81 MB                        |
| Checkpoint Size        | 722.5 MB         | 20.7 MB                         |
| Compression Ratio      | Baseline         | <b>2.28×</b>                    |
| Size Reduction         | Baseline         | <b>97.1% smaller checkpoint</b> |
| FID Score (FFHQ)       | 7.75             | 16.57                           |
| Quantization           | FP32             | INT8 (QNNPACK)                  |
| Backend                | Standard PyTorch | QNNPACK (ARM optimized)         |

Table: Performance comparison: MobileStyleGAN vs EdgeStyleGAN (quantized)

## Key Achievements

- **56.2%** further parameter reduction from MobileStyleGAN
- **34.9×** checkpoint size reduction (722.5 MB → 20.7 MB)

# Model Size Comparison

| Model                                  | Checkpoint Size |
|----------------------------------------|-----------------|
| Original (mobilestylegan_ffhq_v2.ckpt) | 722.5 MB        |
| Pruned 20% (pruned_20.ckpt)            | 35.5 MB         |
| Pruned 40% (pruned_40.ckpt)            | 35.5 MB         |
| Pruned 60% (pruned_60.ckpt)            | 35.5 MB         |
| Pruned 80% (pruned_80.ckpt)            | 35.5 MB         |
| Quantized (qnnpack_quantized.ckpt)     | <b>20.7 MB</b>  |

Table: Checkpoint file sizes for different compression methods

## Key Findings

- Quantization achieves **34.9×** reduction from original checkpoint (722.5 MB → 20.7 MB)
- Quantized model is **58.2%** smaller than pruned models
- Best compression achieved through static quantization with QNNPACK backend

# Quantization Backend Comparison

| Feature                | FBGEMM                   | QNNPACK                       |
|------------------------|--------------------------|-------------------------------|
| CPU architecture       | x86                      | ARM                           |
| Best for               | Servers, PCs             | Phones, tablets, edge devices |
| Batch size             | Large                    | Small (often 1)               |
| Optimized instructions | AVX2, AVX512, VNNI       | ARM NEON                      |
| Quantization types     | Static & dynamic         | Static                        |
| Deployment             | Desktop/server inference | Mobile inference              |

Table: QNNPACK vs FBGEMM quantization backends

## Our Choice: QNNPACK

Selected QNNPACK for ARM-based mobile and edge device deployment, optimized for single-image inference

- Real-time **AR/VR personalization**.
- **Creative tools** on mobile devices.
- AI-generated avatars and filters.
- Edge AI for low-power devices.

# Complete Pipeline: StyleGAN2 $\rightarrow$ MobileStyleGAN $\rightarrow$ EdgeStyleGAN

| Metric                 | StyleGAN2 | MobileStyleGAN         | EdgeStyleGAN     |
|------------------------|-----------|------------------------|------------------|
| Parameters             | 28.27M    | 8.85M                  | 3.88M            |
| Reduction vs StyleGAN2 | Baseline  | 68.7%                  | <b>86.3%</b>     |
| Checkpoint Size        | Large     | 722.5 MB               | <b>20.7 MB</b>   |
| Compression Method     | -         | Knowledge Distillation | Quantization     |
| FID Score              | 2.84      | 7.75                   | 16.57            |
| Upsampling             | Bilinear  | DWT                    | DWT              |
| Convolution            | Standard  | Depthwise              | Depthwise (INT8) |

Table: Complete evolution: StyleGAN2  $\rightarrow$  MobileStyleGAN  $\rightarrow$  EdgeStyleGAN

## Progressive Compression

- **Step 1:** StyleGAN2 (28.27M)  $\rightarrow$  MobileStyleGAN (8.85M) via knowledge distillation
- **Step 2:** MobileStyleGAN (8.85M)  $\rightarrow$  EdgeStyleGAN (3.88M) via quantization
- **Total:** **86.3%** parameter reduction from original StyleGAN2

# Conclusion and Future Work

## Key Achievements

- Explored pruning and quantization (left as future work by MobileStyleGAN authors)
- Successfully compressed MobileStyleGAN from 722.5 MB to 20.7 MB (**34.9× reduction**)
- Reduced parameters from 8.85M to 3.88M (**56.2% reduction**)
- Achieved 2.28× compression ratio with static quantization
- Total **86.3%** parameter reduction from original StyleGAN2

## Future Work

- Combine pruning and quantization for further compression
- Hardware-aware optimization for specific edge devices
- Fine-tuning strategies to improve FID after compression
- Real-time inference benchmarking on mobile hardware

# References



Tero Karras, Samuli Laine, and Timo Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," arXiv:1812.04948, 2018.

<https://arxiv.org/abs/1812.04948>



Sergei Belousov, "MobileStyleGAN: A Lightweight Convolutional Neural Network for High-Fidelity Image Synthesis," arXiv:2104.04767, 2021.

<https://arxiv.org/abs/2104.04767>

**Questions?**  
**Thank You!**